Degrade → Restore → Detect: does repairing a damaged image help a computer find the objects in it?

Oct 24, 2026 · Shourja Banerjee

The project in plain words

What we are asking. Photos that reach a computer vision system are often damaged: grainy from a small sensor, blocky from heavy compression, or too dark. There are many methods that "clean up" such images, and they are usually graded by how close the cleaned picture is to the original, pixel by pixel. This project asks a more practical question: **does cleaning up an image actually help a computer find the objects in it?**

How we test it. Three steps give the project its name:

1. **Degrade.** Take clean photos and damage them in controlled, repeatable ways: seven kinds of damage at five strengths each.
2. **Restore.** Repair them with classical image-processing filters and with a neural network we trained ourselves (DnCNN).
3. **Detect.** Run a standard object detector (YOLOv8n) on the clean, damaged and repaired images and measure how many objects it still finds correctly.

We then compare two scorecards for every repair: how close it is to the original picture (PSNR), and how much it helps the detector (mAP).

What we found.

- **Grainy (noisy) images:** repair helps a lot, and the neural network is clearly best. It wins back about three-quarters of the detection accuracy the noise took away; hand-tuned filters win back a quarter to a half.
- **Compressed images:** the gains are small. A simple edge-aware filter helps a little; the network, which never saw compression during training, does not help.
- **Dark images:** the two scorecards disagree. The method with the best pixel score, contrast boosting (CLAHE), makes detection *worse*, because it brightens the noise along with the picture. The network barely changes the pixel score yet helps detection the most.
- **Why:** looking inside the network shows that it learned, by itself, to treat brightness and colour separately, to smooth more when the image is noisier, and to keep the edges a detector needs. It also breaks in visible ways on kinds of damage it was never trained on.

Why it matters. If a repaired image is going to a machine rather than a person, the repair should be judged by how well the machine does, not only by how close the picture looks to the original.

How to read this report. Sections 2–7 build the damage models and measure classical repair. Sections 8–9 rebuild two core tools, JPEG compression and a neural-network layer, from scratch to show the maths is understood. Section 10 trains the neural network. Section 11 answers the research question, section 12 explains the answer, and sections 13–14 conclude. Each section opens with a one-line summary.

Key terms

Term	What it means here
PSNR	Peak signal-to-noise ratio: how close a repaired image is to the original, pixel by pixel, in decibels (dB). Higher is closer; +6 dB roughly halves the typical pixel error.
SSIM	Structural similarity: a 0–1 score of how alike local patterns look; closer to human judgement than PSNR.
mAP	Mean average precision: the standard object-detection score (0–1). It rewards finding each object with the right label and a well-placed box. We use COCO's mAP@[.5:.95], which averages strict and loose box-overlap thresholds.
σ (sigma)	Strength of the added grain (Gaussian noise) on a 0–255 pixel scale: σ = 15 is light, 25 clearly grainy, 50 very heavy.
Classical filter	A hand-designed smoothing method (Gaussian, median, bilateral, non-local means) with a few settings to tune.
DnCNN	A 17-layer convolutional neural network that looks at a noisy image, predicts the noise, and subtracts it.
Blind	The method is not told how strong the damage is and has to work it out from the image.
Oracle tuning	Settings chosen knowing the true damage strength, an advantage a real system would not have.

Term	What it means here
Residual	What a denoiser removes: its input minus its output.
Receptive field	The patch of input pixels that can affect one output pixel.
Spatial frequency	How quickly brightness changes across the image: low for smooth areas, high for edges, fine texture and noise.
Out of distribution	An input unlike anything seen in training, such as JPEG blocks for a network trained only on grain.
BSD500, COCO	Public image datasets: BSD500 to measure repair quality, COCO (photos with labelled objects) to measure detection.

How the notebooks connect, what each found and what the project is building towards: Project guide

Abstract

This project asks whether restoring a damaged image helps an object detector, or only makes the image look better. We model seven degradations (Gaussian, Poisson and impulse noise, blur, JPEG compression, colour cast and low light) and restore them with four tuned classical filters, colour-space-aware denoising, contrast enhancement, matched inverses (gray-world white balance, Wiener deconvolution) and a blind residual CNN (DnCNN). Restoration quality is measured with PSNR and SSIM on 96 held-out test images from the BSD500 validation set, which contains the standard CBSD68 benchmark. A YOLOv8n detector on 300 COCO val2017 images then tests whether restoration wins back detection accuracy. DnCNN recovers 70–83% of the mAP lost to Gaussian noise, against 27–60% for tuned classical filters. PSNR picks the right restorer for noise and compression, but in low light the highest-PSNR method (CLAHE) is the worst for detection in all four conditions, and DnCNN, which barely changes PSNR there, is the best. Opening up the trained network explains these results: it behaves like a colour-aware filter that adapts to the noise level and keeps up to 1.9× more spatial frequency than the best classical filter; in low light it removes the noise but not the darkness; and on compression and blur, which it never saw in training, it produces streak artifacts along strong edges.

The core maths is implemented from scratch in NumPy: the degradation models, PSNR, a Haar-wavelet noise estimator, Wiener deconvolution, a JPEG codec (within 0.05 dB of libjpeg) and a convolution layer with backpropagation (gradients checked to 7e-10).

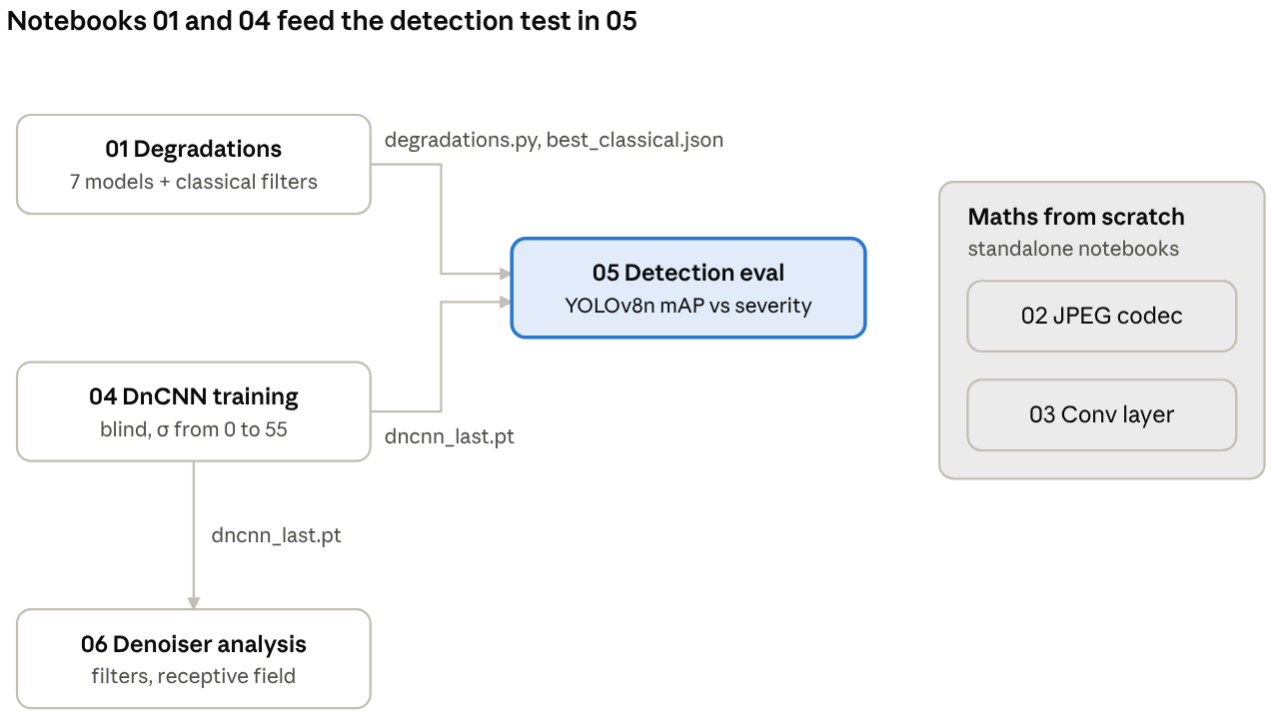
1. Introduction and research question

Real images reach a vision system damaged: sensors add noise, storage adds compression artifacts, and poor lighting or white balance shifts brightness and colour. Restoration methods are usually judged by PSNR or SSIM against a clean reference, but a downstream model such as a detector may need different things than a pixel metric rewards. A denoiser that wins on PSNR by smoothing everything could erase the edges a detector relies on.

Research question: as images get noisier, more compressed or darker, how much does a pretrained detector's mAP drop, how much of that loss does each restoration method recover, and does the best-PSNR method also recover the most mAP?

The work is split into six notebooks. Each one produces files the later ones load, so every result is reproducible from seeded code.

Notebook	Platform	Shows	Produces
01 Degradations and baselines	Kaggle, CPU	noise models, colour and grayscale processing, classical denoising, metrics	`degradations.py`, tuned filter settings, baseline tables
02 JPEG from scratch	Colab, CPU	DCT, quantisation, chroma subsampling, rate–distortion	codec and RD curves
03 Convolution from scratch	Colab, CPU	conv forward and backward passes, gradient checks, im2col	verified NumPy conv layer
04 DnCNN training	Kaggle, GPU	residual learning, blind denoising	trained checkpoint
05 Detection evaluation	Kaggle, GPU	YOLOv8n mAP under degradation and restoration	mAP vs severity
06 Denoiser analysis	Kaggle	learned filters, receptive field, frequency response	interpretability figures



notebook data flow · 6 notebooks

Files written by 01 and 04 are loaded by 05 and 06 as Kaggle inputs; 02 and 03 stand alone.

2. Degradation models (notebook 01)

In short: seven realistic kinds of damage are defined as exact, repeatable formulas, so every repair method is tested on identical damaged images.

Each degradation is a function of the clean image x, a severity s and a seeded random generator. Images are float32 RGB in [0, 1], and clipping to [0, 1] happens once, after the degradation, as a real sensor saturates. Every degradation has five severity levels from mild to severe.

Degradation	Physical cause	Model	Severities
Gaussian noise	sensor read and thermal noise	$y = x + n$, $n \sim N(0, (\sigma/255)^2)$ i.i.d. per pixel and channel	σ = 5, 15, 25, 35, 50
Poisson noise	photon shot noise	$y = \text{Poisson}(p \cdot x) / p$, so $\text{Var}[y] = x / p$	peak p = 100, 50, 20, 10, 5
Salt and pepper	dead or stuck pixels, bit errors	a fraction a of pixels set to 0 or 1, all three channels together	a = 0.01, 0.03, 0.05, 0.1, 0.2
Blur	defocus, camera shake	Gaussian kernel of std σ, reflected borders	σ = 0.5, 1, 1.5, 2, 3 px

Degradation	Physical cause	Model	Severities
JPEG	lossy storage	encode and decode at quality q	q = 90, 70, 50, 30, 10
Colour cast	wrong white balance	channel gains (1 + s, 1, 1 − s) on (R, G, B)	s = 0.1 to 0.5
Low light	under-exposure	x_d = (1 − 0.8ℓ)·x^(1 + 2ℓ), then shot noise (peak 200(1 − ℓ) + 5) and read noise (std 0.01 + 0.04ℓ)	ℓ = 0.2, 0.4, 0.6, 0.8, 0.9

Poisson noise is signal-dependent: its signal-to-noise ratio is √(p·x), so dark regions are relatively noisier. Salt-and-pepper corrupts whole pixels rather than single channels because that is how a dead sensor pixel behaves. The low-light model is our own design: at $\ell = 0$ it is exactly the identity, and both darkening and noise grow monotonically with ℓ.

A test cell checks each model statistically before any experiment runs. For example, Gaussian residuals must have standard deviation σ/255 within 3%, Poisson variance must equal x/p within 5%, and JPEG error must fall as quality rises. All nine checks pass.



Figure 1. Gaussian noise at σ = 5, 15, 25, 35 and 50 on a BSD500 test image.

3. Quality metrics

In short: PSNR and SSIM measure how close a repair is to the original. Neither measures whether a detector can still find objects, which is why section 11 tests that directly.

PSNR is the mean squared error on a log scale, relative to the largest possible signal (MAX = 1 for our images):

```
\mathrm{MSE} = \frac{1}{HWC}\sum_{i}(x_i - \hat{x}_i)^2, \qquad \mathrm{PSNR} = 10\log_{10}\frac{\mathrm{MAX}^2}{\mathrm{MSE}}\ \text{dB}
```

A gain of 6 dB roughly halves the RMS error. We compute it in float64 and define identical images as +∞.

A pitfall we hit and fixed. An early version computed the square of the mean error instead of the mean of the squared errors. Noise has zero mean, so positive and negative errors cancelled and PSNRs of 50–90 dB appeared for visibly noisy images. A constant-offset test could not catch this, because both formulas agree when every error is the same. We added a test with random ±0.1 errors (mean 0, mean square 0.01), which must give exactly 20 dB.

SSIM compares local windows on luminance (means), contrast (variances) and structure (correlation), so it tracks perceived quality better than PSNR. Neither metric measures what a detector needs, which is the question notebook 05 tests.

4. Experiment 1: tuned classical baselines

In short: hand-designed filters repair random noise well (7–11 dB) but cannot repair blur, compression, colour cast or darkness. Their best scores set the bar the neural network has to beat.

Four classical filters set the baseline that DnCNN must beat. Each makes a different assumption about images:

Filter	Idea	Should suit	Fails when
Gaussian	weighted local mean	mild Gaussian noise	edges matter: it blurs them equally
Median	local median, robust to outliers	impulse noise	dense Gaussian noise, thin lines
Bilateral	spatial Gaussian × intensity Gaussian: averages only similar neighbours	noise on strongly edged images	heavy noise, which itself looks like edges
Non-local means	averages 7×7 patches that look alike within a 21×21 window	Gaussian noise, repeating texture	very heavy noise; slow

Protocol. The 100 BSD500 validation images are split into 4 tuning images and 96 test images. For every degradation, severity and filter, a grid search on the tuning images picks the parameters with the best mean PSNR. If no filter beats leaving the image alone, the result is recorded as “none”. The tuned settings are then applied to the test images. Every filter sees the identical noisy image for a given (image, degradation, severity), so comparisons are paired. The test also reports what fraction of images each winning filter improves, not just the mean gain.

Oracle caveat. Parameters are tuned separately for each severity, so the classical filters effectively know the noise level. DnCNN is never told it. The baseline is therefore an upper bound for classical methods, which makes any DnCNN win more convincing.

Blind NL-means. To remove that advantage, σ is estimated from the noisy image itself with Donoho's method: the finest diagonal Haar detail band is almost pure noise in natural images, and its median absolute value divided by 0.6745 estimates σ. The NL-means strength is then set from a fit of the tuned values, $h \approx 0.60 \cdot \hat{\sigma}$. On the 96 test images, $\hat{\sigma}$ averages 7.3 at σ = 5 (image texture counted as noise) and 45.0 at σ = 50 (clipping removes the noise tails). Not knowing σ costs only 0.04–0.31 dB from σ = 15 to 50, but 1.1 dB at σ = 5, where the overestimate makes NL-means smooth away real detail.

Results on the 96 BSD500 val test images, at the middle severity of each degradation.

Degradation	Severity	Noisy PSNR (dB)	Best filter	Best PSNR (dB)	Gain (dB)
Salt and pepper	a = 0.05	18.00	median	28.64	+10.64
Poisson	p = 20	17.56	bilateral	25.15	+7.59
Gaussian	σ = 25	20.53	bilateral	27.56	+7.03
JPEG	q = 50	31.01	bilateral	31.36	+0.35
Colour cast	s = 0.3	20.03	NL-means	20.15	+0.13
Low light	ℓ = 0.6	9.86	Gaussian	9.89	+0.03
Blur	σ = 1.5	25.76	none (bilateral, the best filter, loses 0.13)	25.76	0

Classical filters split cleanly into two groups. On the three **noise** degradations they gain 7–11 dB, and the winner matches each filter's assumption: median on impulse noise (+13.6 dB at a = 0.2, where averaging filters smear the outliers), bilateral on Gaussian and Poisson noise. On the other four, **no filter gains more than 0.6 dB at any severity**: blur makes every filter worse (−1.41 dB at σ = 0.5), and colour cast and low light are brightness and colour errors that smoothing cannot touch. JPEG gains only 0.3–0.5 dB, because its blocking artifacts are structured, not random.

Two results run against intuition. At the heaviest noise (Gaussian σ = 50, Poisson p = 5 and 10), a plain Gaussian blur beats bilateral and NL-means: when noise is as large as the edges, edge-preserving filters preserve the noise too. And for the noise types the gain grows with severity (Gaussian: +3.4 dB at σ = 5 to +8.2 dB at σ = 50), because there is more random error to average away. Bilateral's colour range reached the top of its grid (0.5) at Gaussian σ = 25 and 35, so its tuned score there may be slightly low; at 0.5 on [0, 1] images it already behaves almost like a plain Gaussian blur, so a wider grid is unlikely to change the ranking. Figure 2 shows the full curves.

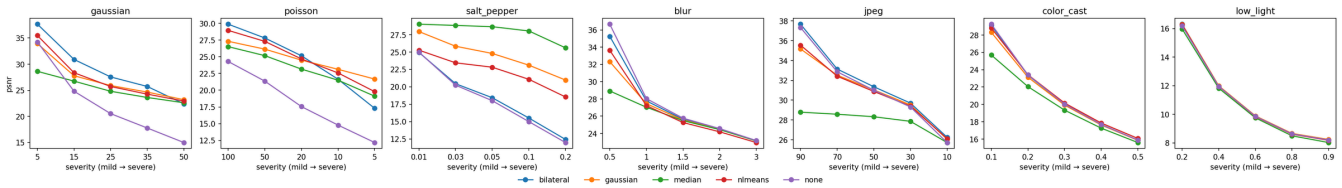


Figure 2. PSNR against severity for every degradation and filter, averaged over the 96 test images.

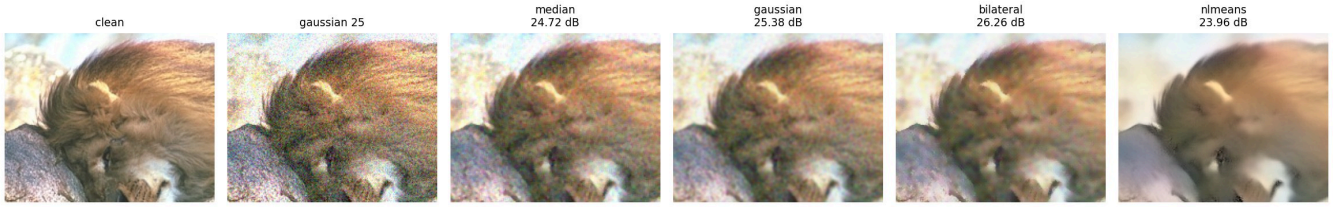


Figure 3. The tuned classical filters at Gaussian σ = 25, each labelled with its PSNR.

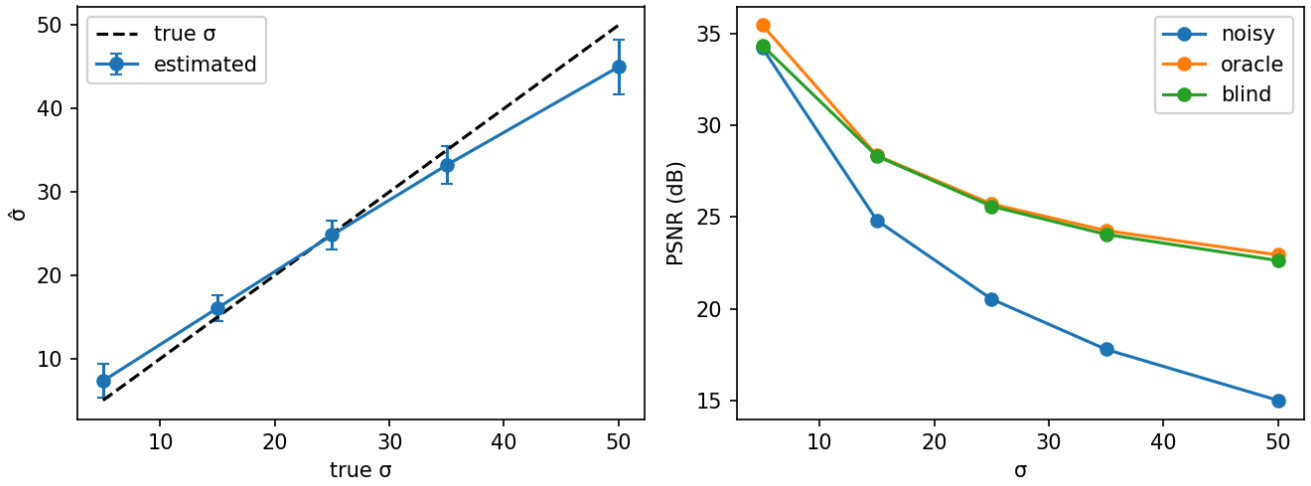


Figure 4. Blind NL-means. Left: σ estimated from the noisy image against the true σ. Right: PSNR of blind and oracle-tuned NL-means.

5. Experiment 2: does the colour space matter for denoising?

In short: smoothing colour strongly and brightness gently beats smoothing red, green and blue equally, because almost all image detail lives in brightness. Section 12 shows the neural network learned the same trick on its own.

Smoothing luma gently and chroma heavily should beat smoothing R, G and B equally, because noise is spread evenly across the YCrCb channels but image detail is not.

Noise in each channel. Each YCrCb channel is a weighted sum w·(R, G, B) of channels carrying independent noise of std σ, so its noise std is $\sigma\|w\|_2$. This holds for any image:

Channel	Weights (R, G, B)	Theory $\sigma\|w\|_2 / \sigma$	Measured
Y	0.299, 0.587, 0.114	0.669	0.667
Cr	0.5, −0.419, −0.081	0.657	0.658
Cb	−0.169, −0.331, 0.5	0.623	0.625

Signal in each channel. In a clean photo, fine detail (image minus a blurred copy) is far larger in Y than in chroma. On test image 0, detail std is 0.083 in Y against 0.004 and 0.006 in Cr and Cb, so at σ = 25 the detail-to-noise ratio is 1.27 in Y but only 0.06–0.10 in chroma. The chroma channels are almost pure noise, and human vision is also less sensitive to colour detail, the same fact JPEG exploits with 4:2:0 subsampling.

Method. A plain Gaussian blur is applied per channel in RGB, YCrCb or Lab with per-channel strengths, then converted back. The filter never changes, so only the colour space and strengths vary. YCrCb (1, 1, 1) must equal RGB (1, 1, 1) exactly, since an affine colour transform commutes with a linear blur; this is a built-in bug check. The reverse configuration, heavy luma and gentle chroma, is the control.

Results. Mean PSNR in dB on the 96 test images.

σ	Noisy	RGB (1, 1, 1)	YCrCb (0.7, 2, 2)	YCrCb (2, 0.7, 0.7)	Lab (0.7, 2, 2)
15	24.82	27.09	29.16	23.88	28.77
25	20.53	25.90	27.42	22.96	26.67
50	15.00	22.89	23.58	20.49	22.08

Gentle luma with heavy chroma beats plain RGB smoothing on 100% of images at σ = 15 and 25 (+2.07 and +1.51 dB) and on 99% at σ = 50 (+0.68 dB), and SSIM rises with it (0.686 to 0.752 at σ = 25). The reverse configuration loses 2.4–3.2 dB against RGB, which confirms that which channel is smoothed is what matters, not how much. Visually, RGB smoothing leaves coloured blotches while YCrCb leaves a fine grey grain that is far less objectionable. Lab trails YCrCb by 0.4–1.5 dB because it is nonlinear, and at σ = 50 it falls below RGB. The advantage shrinks at σ = 50 because a luma strength of 0.7 is too gentle for that much noise.



Figure 5. The same Gaussian blur applied in RGB, YCrCb and Lab at σ = 25, with per-channel strengths and PSNR labelled.

6. Experiment 3: grayscale conversion and contrast enhancement

In short: converting to grey and brightening a dark image both trade one kind of faithfulness for another. Brightening the red, green and blue channels separately badly distorts colour.

Luma vs average. Every grayscale conversion maps 3 numbers to 1, so it must merge some distinct colours; the only choice is which. Plain averaging maps pure red, green and blue all to 0.333. BT.601 luma, Y = 0.299R + 0.587G + 0.114B, gives 0.299, 0.587 and 0.114, matching how bright each primary looks to the eye. The two methods each have colour pairs they cannot tell apart:

Pair (R, G, B)	Luma	Average
(0.9, 0.3, 0.3) vs (0.3, 0.9, 0.3)	0.479 vs 0.652	0.500 vs 0.500 (merged)
(0.9, 0.2, 0.2) vs (0.2, 0.556, 0.2)	0.409 vs 0.409 (merged)	0.433 vs 0.319

Luma merges colours that also look equally bright to people, which is why it is the standard. A grayscale-only detector loses whatever distinction the conversion merged.

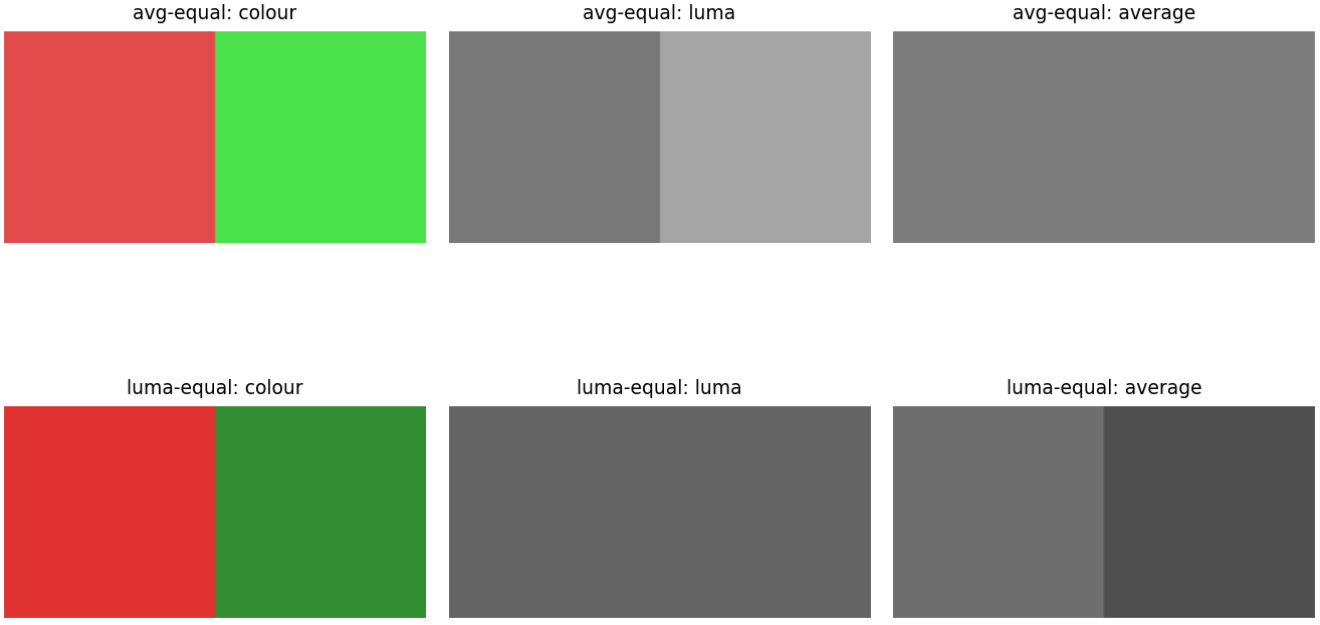


Figure 6. Top: a red/green pair with equal averages, which luma tells apart. Bottom: a pair with equal luma, which the average tells apart.

Histogram equalisation vs CLAHE on low-light images. Global histogram equalisation (HE) maps intensities through the image's own cumulative histogram. CLAHE does this per tile with a clip limit that caps how much contrast, and noise, any tile can gain. Each is applied either to every RGB channel or to the L channel of Lab only. Colour change is measured as the mean distance between clean and output pixels in Lab's (a, b) plane.

Results on the 96 test images at low light $\ell = 0.6$ (clean mean brightness 0.409):

Method	Brightness	Colour shift Δab	PSNR (dB)	SSIM
Degraded	0.111	15.0	9.86	0.214
HE on RGB	0.423	38.6	13.32	0.260
HE on L	0.459	14.9	14.48	0.332
CLAHE on RGB	0.224	23.0	13.14	0.305
CLAHE on L	0.208	15.1	12.83	0.383

Equalising RGB channels separately multiplies the colour shift by 2.6 (15.0 to 38.6, and to 48.0 at $\ell = 0.8$), because each channel gets its own curve and their ratios, which define hue, change; a grey background turns cyan. Working on L leaves colour exactly where the degraded image had it. HE on L restores brightness best and wins on PSNR; CLAHE on L amplifies less noise and wins on SSIM (0.383). PSNR stays at 12.8–14.5 dB even when the image looks good, because equalisation produces a pleasing tone curve rather than the original values. The same ranking holds at $\ell = 0.8$.

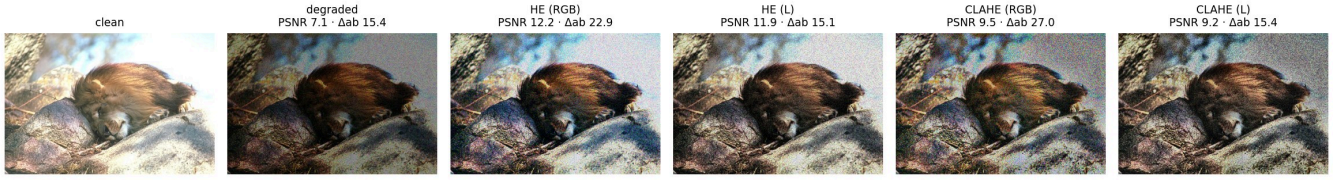


Figure 7. Low light $\ell = 0.6$: degraded input, then HE and CLAHE on RGB and on the L channel.

7. Matched restoration: white balance and deconvolution

In short: blur and colour cast need their own targeted fixes, which noise filters cannot provide. This is the first sign of the project's main theme: a repair works only when it matches the damage.

Denoisers cannot fix blur or colour cast, because those are not noise problems; each needs an inverse that matches its own degradation model.

Gray-world white balance. Assume the scene averages to grey and scale each channel so its mean equals the overall mean. A colour cast is one gain per channel, which gray-world normalises away, so its output is almost the same whatever the cast strength; it is not exactly the same only because strong casts push red above 1 and the clipped values are lost. What remains is the error of the assumption itself. On the 96 test images the colour shift after gray-world stays at 13.4–14.1 for every strength. It beats no correction from s =

0.2 upward (25.82 vs 23.39 dB) and by 7.7 dB at s = 0.5 (23.61 vs 15.89), while the best denoiser gains at most 0.19 dB. At s = 0.1 it loses 2.8 dB, replacing a small cast with its own larger error. Its failure case is test image #15, a blue-dominated scene (channel means 0.22, 0.50, 0.56), which it "corrects" to 15.7 dB even with no cast at all.



Figure 8. Gray-world white balance: clean image, colour cast 0.3, the corrected image, and gray-world applied to the clean image, where its assumption fails.

Wiener deconvolution. Blur multiplies the image spectrum by H, the kernel's Fourier transform. Dividing by H undoes it but amplifies noise wherever H ≈ 0, so Wiener regularises the inverse with K:

```
\hat{X}(u,v) = \frac{H^*(u,v)}{|H(u,v)|^2 + K}\, Y(u,v)
```

It is implemented with NumPy FFTs, with reflection padding to stop ringing at the borders.

PSNR in dB on the 96 test images:

Blur σ	Added noise	No fix	Best denoiser	Wiener	Best K
1	none	28.03	27.77	37.53	1e-4
1	σ = 2	27.81		29.57	1e-2
2	none	24.55	24.48	28.52	1e-4
2	σ = 2	24.45		26.29	1e-2
3	none	23.17	23.13	25.86	1e-4
3	σ = 2	23.10		24.65	1e-2

Without noise, Wiener gains up to 9.5 dB (blur σ = 1), where every denoiser makes the image worse. Adding noise of σ = 2/255, invisible to the eye, moves the best K 100× higher (1e-4 to 1e-2) and cuts the gain to 1.5–1.8 dB; with too small a K, that noise is amplified into coloured speckle. Restoration has to match the degradation model, which is also why a DnCNN trained only on Gaussian noise is expected to struggle on blur and JPEG artifacts (notebook 06).

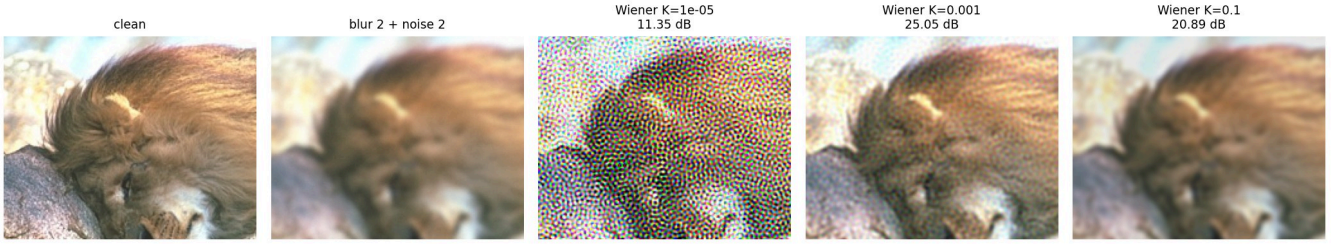


Figure 9. Wiener deconvolution of blur σ = 2 plus noise for three values of K: too small amplifies noise, too large leaves blur.

8–9. Compression and CNN maths from scratch

In short: to show we understand two key tools rather than only calling a library, we rebuilt the JPEG compressor and a neural-network convolution layer in plain NumPy and checked both against trusted references.

Both notebooks reimplement a standard tool in NumPy and check it against an independent reference: OpenCV's libjpeg codec for notebook 02, SciPy and numerical gradients for notebook 03. They ran on the BSD500 images on a CPU.

8. JPEG from scratch (notebook 02)

The codec implements the lossy core of baseline JPEG: RGB to YCbCr, 4:2:0 chroma subsampling, 8×8 blocks, the 2-D DCT as $F = C \cdot B \cdot C^{T}$ with an orthonormal C, quantisation with the standard tables scaled by quality, and zigzag ordering. File size is estimated as the entropy of JPEG's own symbols: DC differences between blocks, and (zero-run, size) pairs with end-of-block markers for AC, plus the amplitude bits. Checks confirm that C is orthonormal, matches SciPy's DCT, inverts exactly and preserves energy.

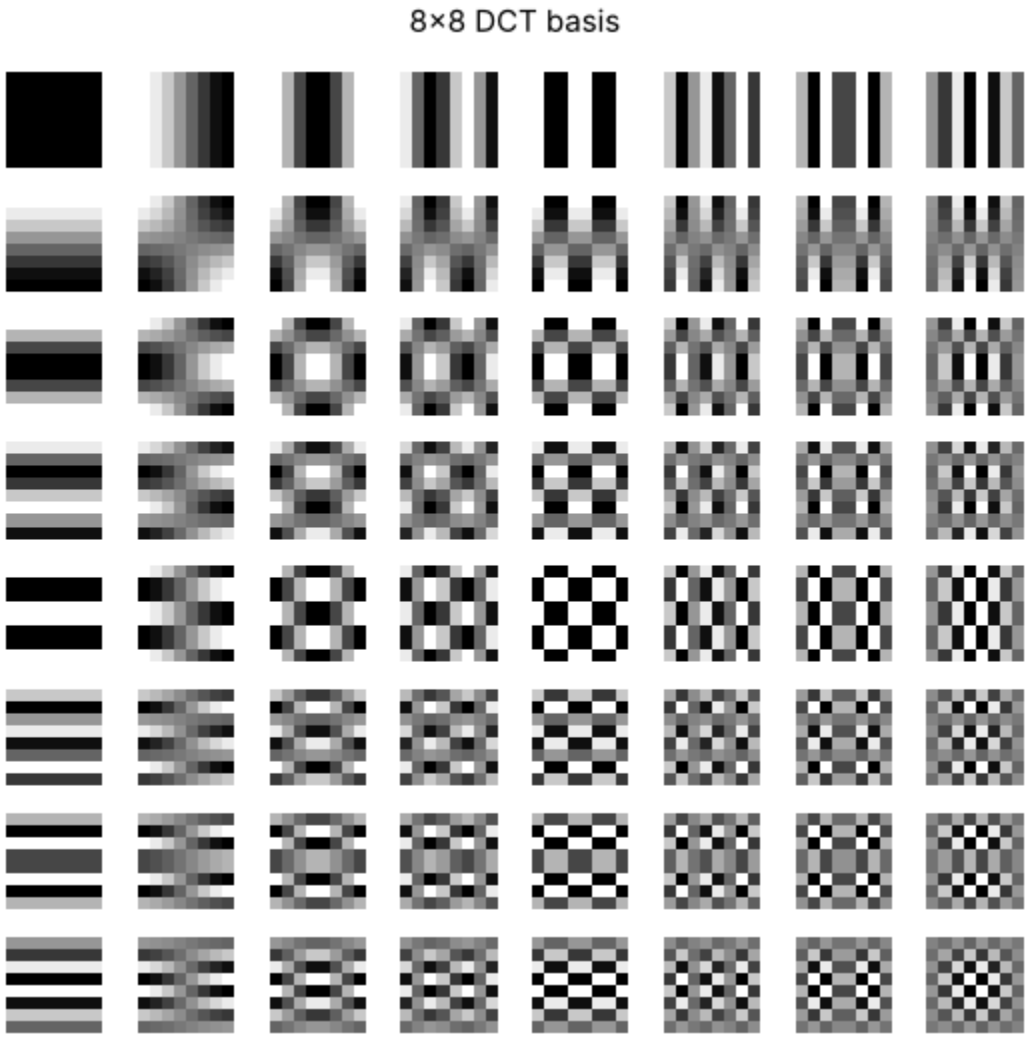


Figure 10. The 64 DCT basis patterns of an 8×8 block.

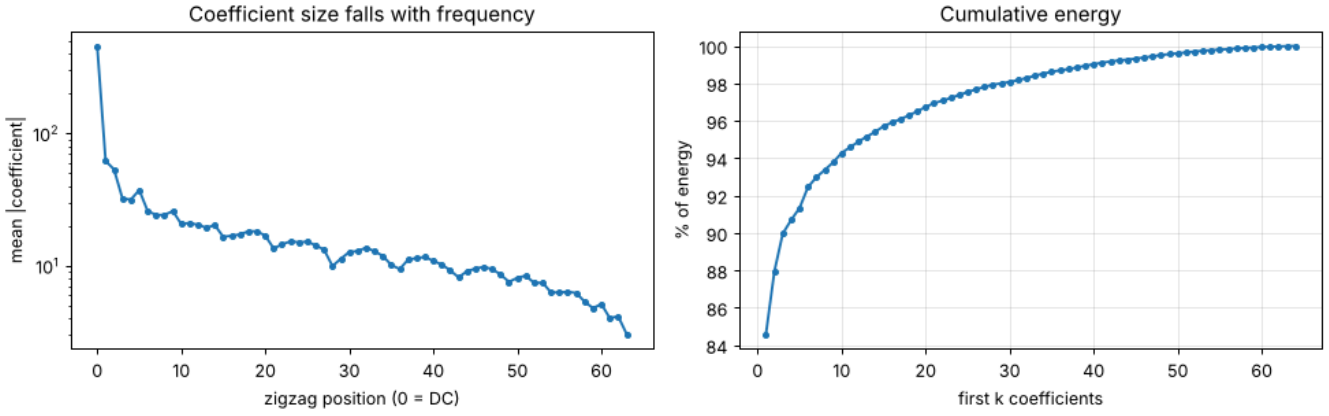


Figure 11. Mean coefficient size by zigzag position (left) and cumulative share of energy in the first k coefficients (right).

At the same quality setting our PSNR is within 0.05 dB of libjpeg up to quality 90, so the transform and quantisation match the standard. Results on one validation image (480×320):

Quality	Ours (bits/pixel)	OpenCV (bits/pixel)	Ours PSNR (dB)	OpenCV PSNR (dB)
10	0.378	0.492	23.22	23.23
30	0.967	1.062	26.27	26.26
50	1.410	1.495	27.90	27.90
70	2.003	2.087	29.85	29.84
90	3.760	3.929	34.87	34.82
95	4.936	5.215	48.85	49.78

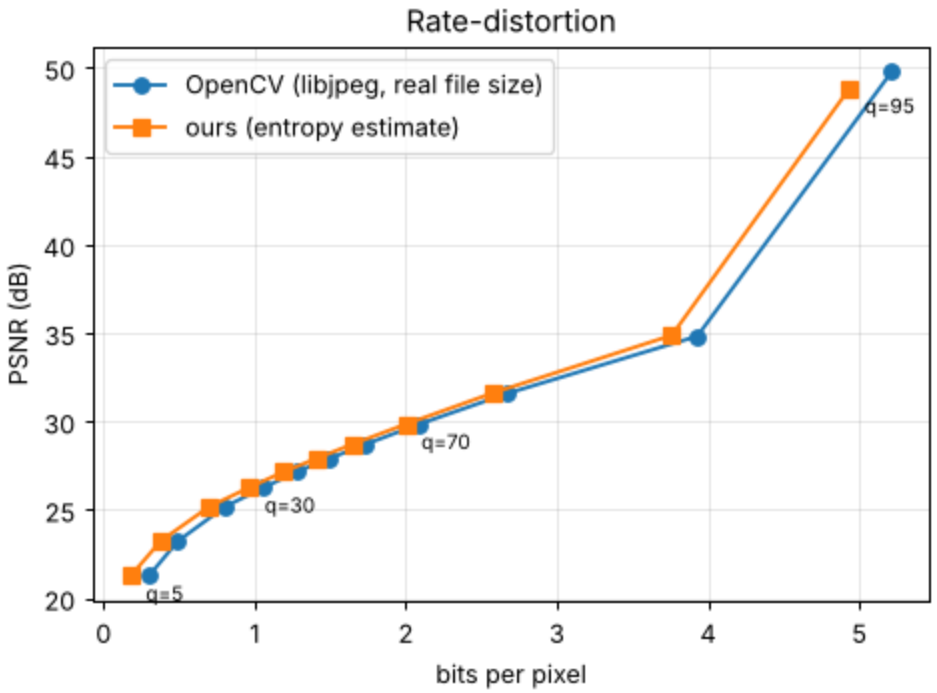


Figure 12. Rate-distortion: our codec (estimated size) against OpenCV's libjpeg (real file size).

- **Size.** Over 20 images our estimate is 0.86× OpenCV's real file size. It is an idealised lower bound: it skips libjpeg's fixed Huffman tables and the roughly 600-byte file header, so it is not a better codec. A naive estimate that ignores run-length coding came out 18% *larger* than libjpeg at quality 50 (1.763 vs 1.495 bits/pixel), which shows that coding the runs of zeros is where much of JPEG's compression comes from.
- **Energy compaction.** The DC coefficient holds 84.6% of the luma energy and the first 15 of 64 coefficients hold 95.7%. At quality 50, 73.8% of luma coefficients quantise to exactly zero.
- **Artifacts.** At quality 10, pixel steps across 8×8 block boundaries average 19.1 against 8.9 inside blocks (2.1×), while the original shows no block structure (15.5 vs 15.8). That is the blocking seen in the figures, because each block is quantised independently.

- **Chroma subsampling.** 4:2:0 saves 8–10% of bits for 0.2–0.65 dB. How chroma is upsampled matters more: copying each value into a 2×2 block costs 4.9 dB at quality 95, against 0.27 dB with bilinear interpolation. That was the cause of an initial 5.5 dB gap to libjpeg at quality 95, which uses smoothed upsampling.



Figure 13. Quality 10: original, our codec, OpenCV, and the blocking measure (mean step across block edges against inside blocks).

9. A convolution layer from scratch (notebook 03)

The notebook derives the forward pass and all three gradients, then implements them twice: a readable version that loops over output positions, and an im2col version that turns the whole layer into one matrix multiply.

- **Forward pass.** It matches SciPy's 2-D cross-correlation to 1e-10 on three cases covering kernel sizes 3 and 5, strides 1 and 2, and padding 0 to 2.
- **Backward pass.** Analytic gradients for input, weights and bias match central-difference numerical gradients with a relative error of at most 7e-10, at strides 1 and 2. The check has teeth: a backward pass with the kernel orientation deliberately wrong gives an error of 0.71.
- **The rotated kernel.** For stride 1 the input gradient equals a full cross-correlation of the upstream gradient with the 180°-rotated kernel, and equally a full convolution with the unrotated one; both identities are verified numerically.
- **im2col.** It is 248× faster than element-by-element Python loops, and 2× faster than the position-loop version, which already batches the inner sums. The price is memory: every pixel is copied $k^2 = 9$ times, so a batch of 8×16×32×32 inputs needs a 9.4 MB matrix.
- **A denoiser built only from this layer.** Three layers (1 → 16 → 16 → 1 channels) trained to predict the noise, 400 Adam steps on 32×32 grayscale BSD500 training patches at $\sigma = 25$, took 23 s on a CPU. The loss fell from 0.0080 to 0.0021 (predicting zero scores 0.0096), and a held-out validation crop went from 20.65 to 25.43 dB. The same residual-learning idea, scaled to 17 layers and 3 colour channels, is the DnCNN of section 10.

The notebook also compares against PyTorch's conv2d, autograd and conv_transpose2d. Those cells were skipped here because PyTorch was not installed; they run automatically wherever it is.

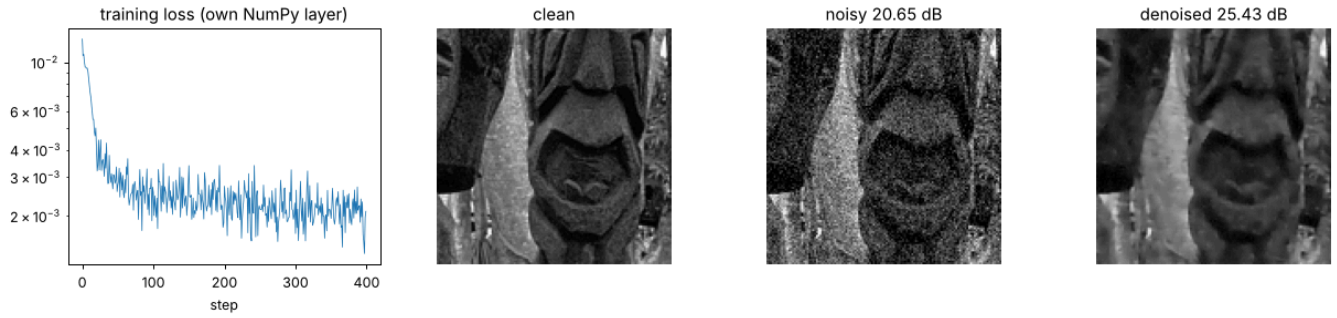


Figure 14. The 3-layer NumPy denoiser: training loss and a σ = 25 example (20.65 → 25.43 dB).

10. Learned restoration: blind DnCNN (notebook 04)

In short: a 17-layer neural network trained to predict and subtract noise beats the best hand-tuned filter by 3–4.5 dB, without being told how noisy the image is, and matches the published model.

DnCNN (Zhang et al., 2017) predicts the noise in an image rather than the clean image: given a noisy input y, it outputs $\hat{v} = f(y)$, and the restored image is $y - \hat{v}$. Learning the residual is easier because the identity map is already nearly right, and the noise has a much simpler structure than natural images.

Training data. The BSDS500 train and test folders, 400 colour images. The 100 val images, which contain CBSD68, are held out for testing; a check by file name and pixel hash found 0 shared images. Each image is resized to 100%, 90%, 80% and 70%, and 40×40 patches are cut with stride 20, giving about 381,000 patches (1.83 GB as uint8) for 400 images.

Noise on the fly. Each time a patch is loaded it gets a random rotation or flip (one of 8) and fresh Gaussian noise with σ drawn uniformly from [0, 55], so the model never sees the same pair twice and must infer σ itself (blind denoising). Noisy patches are clipped to [0, 1] to match the evaluation pipeline and real sensors, unlike the original paper. Clipping shrinks the effective noise, by about 12% at σ = 50 in our checks, so the training target is computed after clipping.

Architecture. 17 convolutional layers, all 3×3 with padding 1 so output size equals input size:

Layers	Structure	Parameters
1	Conv 3 → 64, ReLU	1,792

Layers	Structure	Parameters
2–16	Conv 64 → 64 without bias, BatchNorm, ReLU	15 × 36,992 = 554,880
17	Conv 64 → 3, no activation	1,731

Total: 558,403 parameters. The receptive field is 1 + 2 × 17 = 35 pixels. The last layer has no ReLU because noise is negative about half the time, and convolutions before BatchNorm drop their bias because BatchNorm's own shift replaces it.

Training setup. MSE loss between predicted and true noise, Adam with learning rate 1e-3, divided by 10 at 60% and 85% of 50 epochs, batch size 128, mixed precision on a Kaggle T4, run as a background commit with a checkpoint every epoch.

Results on the 96 BSD500 validation test images.

σ	Noisy PSNR (dB)	Best classical, oracle (dB)	DnCNN PSNR (dB)	DnCNN over classical (dB)	Noisy SSIM	DnCNN SSIM
15	24.82	30.90 (bilateral)	33.76	+2.86	0.600	0.931
25	20.53	27.56 (bilateral)	31.05	+3.49	0.424	0.882
50	15.00	23.20 (Gaussian)	27.69	+4.49	0.220	0.778
65 (outside training range)	13.09	not tested	26.16		0.163	0.720
75 (outside training range)	12.11	not tested	24.56		0.137	0.637

DnCNN beats the best tuned classical filter by 2.9–4.5 dB, and the gap widens as noise grows. This holds even though the classical filters were tuned separately for each σ (the oracle advantage of section 4) and DnCNN was never told σ. Both columns come from the same noisy images, so the comparison is paired. Against blind NL-means (28.33 / 25.59 / 22.63 dB), the classical method that, like DnCNN, must infer σ itself, the margin is 5.1–5.5 dB.

DnCNN improves every one of the 96 images at every noise level; the smallest single-image gain is 5.2 dB (σ = 15) and the largest 21.8 dB (σ = 50). The noisy PSNRs match notebook 01's to two decimals, which confirms both notebooks used identical noisy images.



Figure 15. DnCNN at σ = 25: clean crop, noisy (20.85 dB) and denoised (29.86 dB).

Against the paper. The DnCNN paper reports 33.89, 31.23 and 27.92 dB for its blind colour model on CBSD68 at σ = 15, 25 and 50. Ours is within 0.1–0.2 dB at each level. The setups differ slightly (CBSD68 is 68 of our 100 images, and our noise is clipped), so this shows our model matches the published one rather than beating it.

Training. The loss fell below the predict-zero baseline (0.0137) within the first epoch and levelled off at 0.00104 by about epoch 35, equivalent to 29.8 dB averaged over σ from 0 to 55. Most of the gain came in the first 10 epochs. Each epoch took about 275 s on a Kaggle T4, 3.8 hours in total. A small loss bump at epoch 30, where the learning rate dropped 10×, recovered within one epoch.

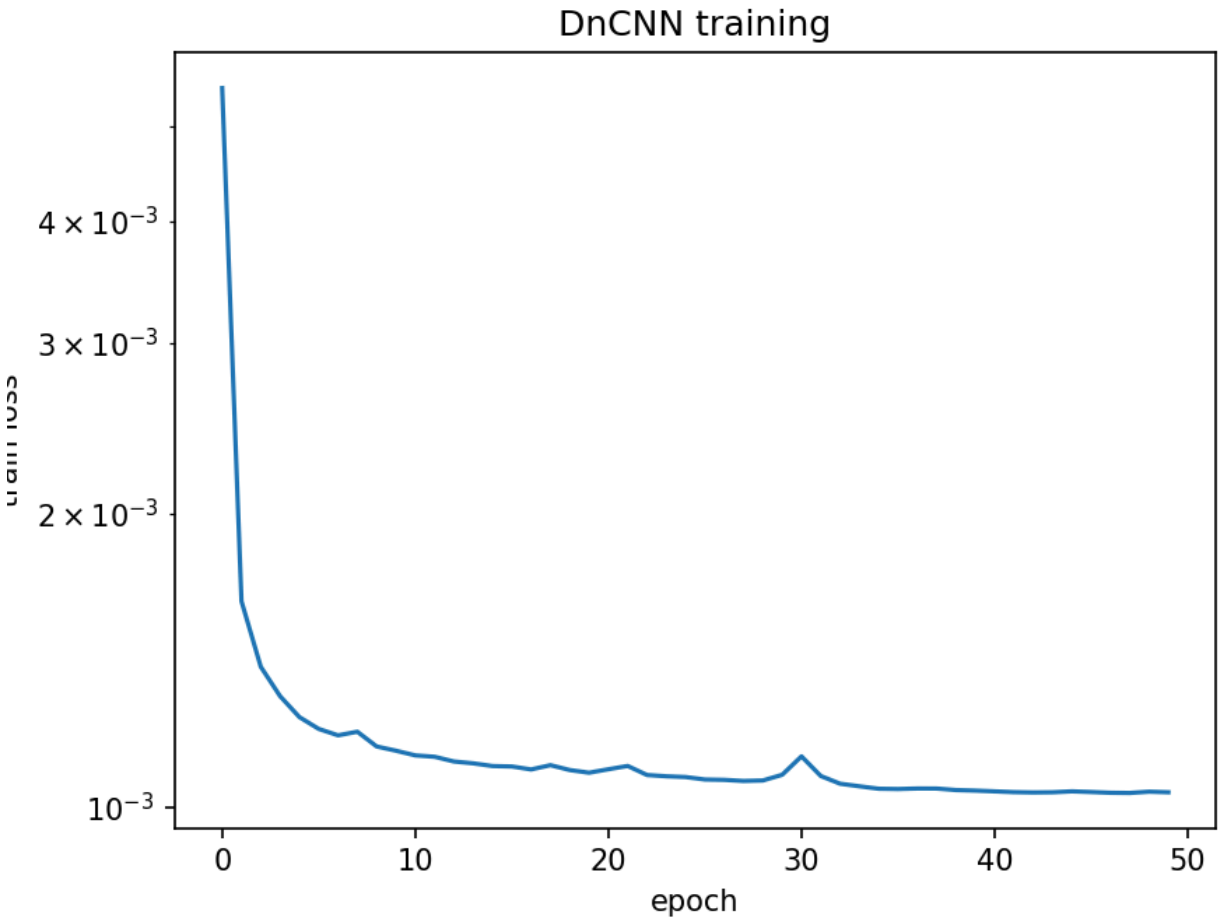


Figure 16. DnCNN training loss over 50 epochs.

Beyond the training range. At σ = 65 and 75 the model still gains about 13 dB, so it degrades gradually instead of failing. At σ = 65 the noise is gone from smooth areas, but fine texture such as fur is visibly smoothed away.

11. Does restoration help detection? (notebook 05)

In short: this section answers the research question. Restoration helps detection when it removes the kind of damage the detector is sensitive to. The pixel score (PSNR) points to the right repair for noise and compression, but for dark images it points to the worst one.

Setup. A pretrained YOLOv8n detector was scored with the official COCO mAP@[.5:.95] on a fixed random subset of 300 COCO val2017 images (2,035 objects). Clean mAP is 0.387, in line with YOLOv8n's published 0.373 on the full validation set. Each image was degraded with Gaussian noise, JPEG or low light at four severities, using notebook 01's functions and seeds, then restored by each method: none, the classical filter notebook 01 tuned for that exact condition, DnCNN (trained only on Gaussian noise) and, for low light, CLAHE on L. Every method saw the identical degraded image.

Predictions. Before running the test we wrote down five hypotheses, based on sections 4–10:

- **H1.** mAP falls steadily as the damage gets stronger.
- **H2.** On Gaussian noise both restorers win back much of the lost mAP, DnCNN more.
- **H3.** On JPEG little is recovered, and DnCNN, never trained on block artifacts, may even hurt.
- **H4.** In low light, CLAHE (local contrast boosting) helps more than any denoiser.
- **H5.** In at least one condition, the restorer with the best PSNR is not the one with the best mAP.

To compare conditions we report the **share of lost mAP recovered**, (restored − damaged) / (clean − damaged). 100% means the repair brought detection back to the clean level, 0% means it did not help, and a negative value means it made detection worse.

mAP@[.5:.95] on the 300 images (clean: 0.387; best per row in bold):

Degradation	Severity	No restoration	Classical	DnCNN	CLAHE on L
Gaussian	σ = 15	0.319	0.359	**0.375**	
Gaussian	σ = 25	0.241	0.305	**0.354**	
Gaussian	σ = 35	0.181	0.264	**0.326**	
Gaussian	σ = 50	0.088	0.169	**0.304**	
JPEG	q = 70	0.376	**0.378**	**0.378**	
JPEG	q = 50	0.349	**0.366**	0.348	
JPEG	q = 30	0.309	**0.346**	0.305	
JPEG	q = 10	0.125	**0.231**	0.121	
Low light	ℓ = 0.4	0.274	0.286	**0.325**	0.250
Low light	ℓ = 0.6	0.207	0.226	**0.283**	0.174
Low light	ℓ = 0.8	0.121	0.134	**0.213**	0.095
Low light	ℓ = 0.9	0.056	0.088	**0.159**	0.044

Image — mAP@[.5:.95] against severity for each restorer on Gaussian noise, JPEG and low…

Figure 17. mAP@[.5:.95] against severity for each restorer on Gaussian noise, JPEG and low light; the dashed line is clean mAP.

H1, mAP falls steadily with damage: confirmed. Every degradation lowers mAP at every step, to 0.088 at Gaussian σ = 50, 0.125 at JPEG quality 10 and 0.056 at low light 0.9, from 0.387 clean.

H2, both restorers help on Gaussian noise and DnCNN more: confirmed. DnCNN wins back 70–83% of the lost mAP at every σ, while the tuned classical filter's share falls from 60% at σ = 15 to 27% at σ = 50. At σ = 50 DnCNN keeps 0.304, almost double the classical filter's 0.169.

H3, little is recovered on JPEG and DnCNN may hurt: half confirmed. DnCNN, which never saw block artifacts, gives no benefit (between −5% and +23% of the lost mAP, the larger value at q = 70 where almost nothing was lost) and lowers PSNR by 1–2 dB below the compressed input. The classical bilateral filter, however, recovers 41–48% of the lost mAP at quality 10–50 while raising PSNR by only 0.4–0.5 dB: smoothing the 8×8 block edges removes false edges the detector would otherwise respond to.

H4, CLAHE helps most in low light: rejected. CLAHE makes detection worse than doing nothing in all four low-light conditions (−4% to −21% of the lost mAP), because it amplifies the shot and read noise along with the brightness. DnCNN, although low light is outside its training data, is the best method at every level, recovering 31–45%.

H5, PSNR does not always pick the best restorer for detection: confirmed for low light. Across all 40 runs PSNR and mAP correlate strongly (Spearman 0.82), but that is driven by severity, which lowers both. What matters for choosing a method is the ranking within one condition. There, PSNR and mAP order every pair of restorers the same way on Gaussian noise and JPEG (0 of 20 pairs reversed), but reverse 17 of 24 pairs in low light. In every low-light condition the highest-PSNR method, CLAHE, is the worst for detection, and DnCNN, the best for detection, leaves PSNR within 0.1 dB of the unrestored image while raising mAP by 0.05–0.10. PSNR there is dominated by the brightness gap, which no denoiser closes, while the detector cares about the noise, which DnCNN removes. On JPEG the order agrees but the scale does not: a 0.5 dB PSNR gain carried an 85% relative gain in mAP at quality 10.

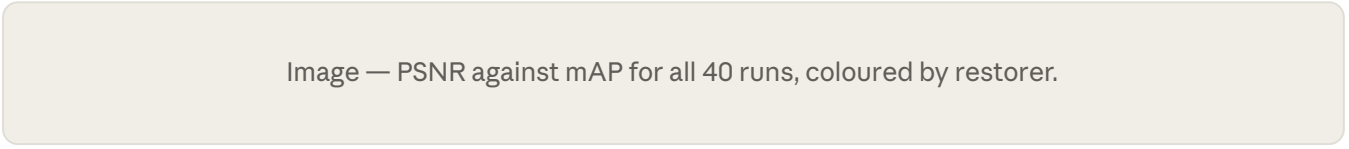


Figure 18. PSNR against mAP for all 40 runs, coloured by restorer.

12. What the denoiser learned (notebook 06)

In short: we looked inside the trained network to see how it works. It behaves like a smart smoothing filter that handles colour separately, adapts to the noise level and keeps edges, which explains both its wins and its failures in section 11.

Notebook 06 opens up the trained DnCNN to explain the detection results in section 11. It uses the 96 BSD500 test images (20 for the residual probes, 10 for the spectra) and the per-image detection results at σ = 50 from notebook 05.

First-layer filters. Only 9 of the 64 first-layer 3×3 filters closely resemble a textbook kernel (cosine similarity of at least 0.8): 6 Sobel edge detectors, 1 Laplacian, 1 Gaussian and 1 identity. Most of the rest work across colour: the typical filter puts 79% of its weight on differences between the R, G and B channels, and 43 of the 64 are mostly colour-opponent (for example, red minus green). The network was never told about colour spaces, yet it learned the structure experiment 2 found by hand: separate brightness from colour, and treat the colour differences, which carry little image detail, differently.

Image — The 64 first-layer filters of DnCNN (RGB, each normalised).

Figure 19. The 64 first-layer filters of DnCNN (RGB, each normalised).

Receptive field. Each output pixel can in principle be affected by a 35 × 35 patch of input pixels (±17 px). To see which pixels actually matter, we measured how much a change in each input pixel changes one output pixel (the gradient). 90% of that influence comes from within ±7 px at σ = 15 and 25, widening to ±10 px at σ = 50, and nothing outside the 35 × 35 patch has any influence, as the architecture requires. So the network mostly looks close by, and looks further when the noise is heavier. The oracle-tuned classical filters do the same by picking larger windows at higher σ, but they are told the noise level; DnCNN works it out from the image.

Image — Effective receptive field: gradient of one output pixel with respect to the inp…

Figure 20. Effective receptive field: gradient of one output pixel with respect to the input at σ = 15, 25 and 50.

Frequency response. An image holds coarse structure (low spatial frequencies, such as smooth areas) and fine detail (high frequencies: edges, texture, and also noise). A denoiser has to remove high-frequency noise without removing too much high-frequency detail. Dividing the power spectrum of the output by that of the noisy input shows how much of each frequency survives. The network is non-linear, so this is an average over 10 images, not a fixed filter. The frequency at which half the power survives (higher means more fine detail kept):

σ	DnCNN (cycles/pixel)	Best classical (cycles/pixel)
15	0.355	0.293 (bilateral)
25	0.254	0.137 (bilateral)
50	0.152	0.098 (Gaussian)

Both move their cutoff down as noise grows, but DnCNN's cutoff is up to 1.9× higher (σ = 25), and at σ = 50 it passes about six times more of the highest frequencies than the Gaussian filter. Those frequencies carry edges and texture, which is the most likely reason DnCNN wins back most of the mAP lost to Gaussian noise (H2). At σ = 15 the bilateral filter keeps slightly more of the very highest frequencies (above about 0.38 cycles/pixel), but there it is mostly passing noise.

Image — Average transfer function on luma: DnCNN (solid) against the best classical fil…

Figure 21. Average transfer function on luma: DnCNN (solid) against the best classical filter (dashed). The dotted line marks half power.

What the network removes. The predicted residual is the noise DnCNN thinks it sees. Averages over 20 test images:

Input	PSNR in (dB)	PSNR out (dB)	Change (dB)	Edge leak
Gaussian σ = 25	20.55	31.11	+10.56	−0.02
Salt and pepper 0.05	17.92	18.15	+0.24	−0.01
JPEG q = 10	25.65	24.56	−1.08	0.05
Blur σ = 2	24.31	22.58	−1.73	0.05
Low light 0.8	8.68	8.58	−0.10	0.05

Edge leak is the correlation between the size of the residual and the edge strength of the clean image. Near 0 means the network is not removing image structure.

- **Gaussian noise.** The residual correlates 0.935 with the true noise, and its standard deviation is 21.4 against 23.0 for the true noise (0–255 scale). DnCNN removes noise, not edges (leak −0.02), and it slightly under-removes rather than over-smoothing.
- **Salt and pepper.** Almost untouched (+0.24 dB, against about +10 dB for the median filter in section 4). Pixels forced to 0 or 1 look nothing like the Gaussian noise it was trained on, so it treats them as image content.

- **JPEG and blur.** PSNR falls. The residual is near zero almost everywhere but has a few large, saturated spikes along strong edges, which show up in the output as blue and yellow streaks. These are out-of-distribution failures: sharp block boundaries and smooth blurred edges are patterns the network never saw, and a few activations blow up. The spikes explain the very large residual standard deviation (52 on JPEG and 157 on blur, against 23 on Gaussian noise), and they are the same kind of numerical blow-up that made half-precision (fp16) inference produce invalid values on JPEG inputs in notebook 05, which is why DnCNN runs in full precision there. These artifacts are why DnCNN gave no detection benefit on JPEG (H3).
- **Low light.** The residual looks like noise, and the output is cleaner but just as dark. DnCNN removes the noise and leaves the brightness alone, so PSNR, which is dominated by the brightness gap, does not move (−0.10 dB), while the detector, which section 11 shows is more sensitive to the noise, gains. This is the mechanism behind the H5 reversal: CLAHE closes the brightness gap (higher PSNR) but amplifies the noise (lower mAP); DnCNN does the opposite.

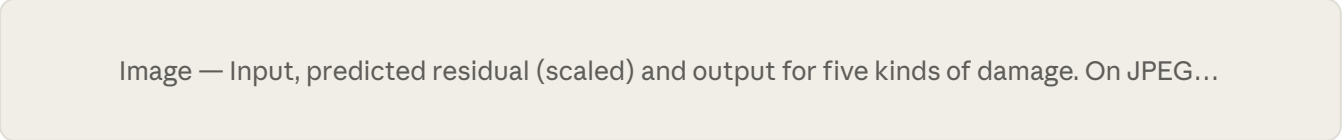


Figure 22. Input, predicted residual (scaled) and output for five kinds of damage. On JPEG and blur the residual is a few saturated streaks along edges.

Where higher PSNR still loses. At Gaussian σ = 50, DnCNN has higher PSNR than the best classical filter on 100% of the 300 COCO images, yet lower AP on 40 of them (13%). In the worst cases DnCNN's output is clean but over-smoothed in flat, low-contrast regions such as fur, cushions and background, and YOLO misses objects or draws smaller boxes (AP 0.70 → 0.24, 0.80 → 0.37 and 0.37 → 0.05), while the noisier classical output keeps enough texture to detect them. The zebra image shows the opposite case: the classical filter blurs the stripes and finds nothing (AP 0.00), while DnCNN keeps them (AP 0.96). PSNR rewards removing as much noise as possible; the detector needs the texture that identifies an object, and at very high noise the two can disagree even between images where one method wins on average.

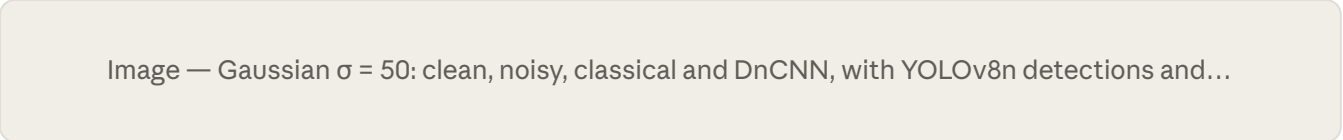


Figure 23. Gaussian σ = 50: clean, noisy, classical and DnCNN, with YOLOv8n detections and per-image AP. In the first three rows DnCNN has the higher PSNR but the lower AP.

In short, DnCNN acts as a learned, colour-aware, noise-adaptive smoother that keeps more edges than any fixed filter, but only for the kind of damage it was trained on.

13. Conclusion

Restoration does help a pretrained detector, but only when it removes the damage the detector is sensitive to; looking better by PSNR is not enough.

- **Noise** is where restoration matters most. DnCNN recovers 70–83% of the mAP lost to Gaussian noise, against 27–60% for tuned classical filters, because it adapts to the noise level and keeps more edges (section 12).
- **Compression** gains are modest. The bilateral filter recovers 41–48% of the lost mAP at quality 10–50 by smoothing block edges; DnCNN, never trained on JPEG, recovers none and adds edge artifacts.
- **Low light** is where PSNR misleads. CLAHE has the best PSNR and the worst mAP in every condition; DnCNN has the best mAP while leaving PSNR almost unchanged.
- The components built from scratch match their references: the JPEG codec within 0.05 dB of libjpeg, the convolution gradients to 7e-10, and DnCNN within 0.2 dB of the published results.

The practical lesson is to choose a restorer by the metric of the task that follows it, or at least to check one. PSNR is a reliable guide when the damage is additive noise or compression, but not when restoration changes brightness or contrast.

14. Limitations

- One detector (YOLOv8n) on 300 COCO images. Small mAP differences (around 0.01, such as the JPEG q = 70 row) were not tested for statistical significance.
- All degradations are synthetic. Real sensor noise is signal-dependent and spatially correlated, and real low-light images are not a simple scaling plus noise.
- DnCNN was trained only on Gaussian noise, so its JPEG, blur and low-light results measure how it generalises, not what a learned restorer trained for those damages could do. A model trained on JPEG artifacts would probably change the H3 verdict.
- The classical filters were tuned with an oracle that knows the severity, which favours them; DnCNN's margin over them is therefore conservative.
- The interpretability measures are averages over 10–20 images, and only the first layer's filters were inspected directly.
- The JPEG size estimate is an idealised lower bound (0.86× the real file size).

15. Future work and reproducibility

- Train DnCNN on JPEG artifacts and low-light images as well as noise, then rerun section 11. This would show whether the negative JPEG result (H3) comes from the network or from its training data.
- Repeat the detection test with a second detector and more COCO images, with confidence intervals on mAP.
- Test on real noisy and low-light photographs instead of synthetic damage.
- Train a restorer whose loss rewards detection directly, rather than pixel closeness.

Reproducibility. All randomness goes through seeded NumPy or PyTorch generators, and each noisy image is seeded by its (image, degradation, severity) indices, so every method sees the same input. Notebook 01 writes `degradations.py` and `best_classical.json`, which notebooks 04 and 05 load as Kaggle inputs. The COCO subset ids are saved with the detection results.